

C. 解释为什么这个优化保持了期望的行为, 或者给出一个例子说明它产生了与使用较少优化的代码不同的结果。

使用了这最后的变换, 至此, 对于每个元素的计算, 都只需要 1.25~5 个时钟周期。比起最开始采用优化时的 9~11 个周期, 这是相当大的提高了。现在我们想看看是什么因素在制约着代码的性能, 以及可以如何进一步提高。

5.7 理解现代处理器

到目前为止, 我们运用的优化都不依赖于目标机器的任何特性。这些优化只是简单地降低了过程调用的开销, 以及消除了一些重大的“妨碍优化的因素”, 这些因素会给优化编译器造成困难。随着试图进一步提高性能, 必须考虑利用处理器微体系结构的优化, 也就是处理器用来执行指令的底层系统设计。要想充分提高性能, 需要仔细分析程序, 同时代码的生成也要针对目标处理器进行调整。尽管如此, 我们还是能够运用一些基本的优化, 在很大一类处理器上产生整体的性能提高。我们在这里公布的详细性能结果, 对其他机器不一定有同样的效果, 但是操作和优化的通用原则对各种各样的机器都适用。

为了理解改进性能的方法, 我们需要理解现代处理器的微体系结构。由于大量的晶体管可以被集成到一块芯片上, 现代微处理器采用了复杂的硬件, 试图使程序性能最大化。带来的一个后果就是处理器的实际操作与通过观察机器级程序所察觉到的大相径庭。在代码级上, 看上去似乎是一次执行一条指令, 每条指令都包括从寄存器或内存取值, 执行一个操作, 并把结果存回到一个寄存器或内存位置。在实际的处理器中, 是同时对多条指令求值的, 这个现象称为指令级并行。在某些设计中, 可以有 100 或更多条指令在处理中。采用一些精细的机制来确保这种并行执行的行为, 正好能获得机器级程序要求的顺序语义模型的效果。现代微处理器取得的了不起的功绩之一是: 它们采用复杂而奇异的微处理器结构, 其中, 多条指令可以并行地执行, 同时又呈现出一种简单的顺序执行指令的表象。

虽然现代微处理器的详细设计超出了本书讲授的范围, 对这些微处理器运行的原则有一般性的了解就足够能够理解它们如何实现指令级并行。我们会发现两种下界描述了程序的最大性能。当一系列操作必须按照严格顺序执行时, 就会遇到延迟界限(latency bound), 因为在下一条指令开始之前, 这条指令必须结束。当代码中的数据相关限制了处理器利用指令级并行的能力时, 延迟界限能够限制程序性能。吞吐量界限(throughput bound)刻画了处理器功能单元的原始计算能力。这个界限是程序性能的终极限制。

5.7.1 整体操作

图 5-11 是现代微处理器的一个非常简单化的示意图。我们假想的处理器设计是不太严格地基于近期的 Intel 处理器的结构。这些处理器在工业界称为超标量(superscalar), 意思是它可以在每个时钟周期执行多个操作, 而且是乱序的(out-of-order), 意思就是指令执行的顺序不一定要与它们在机器级程序中的顺序一致。整个设计有两个主要部分: 指令控制单元(Instruction Control Unit, ICU)和执行单元(Execution Unit, EU)。前者负责从内存中读出指令序列, 并根据这些指令序列生成一组针对程序数据的基本操作; 而后者执行这些操作。和第 4 章中研究过的按序(in-order)流水线相比, 乱序处理器需要更大、更复杂的硬件, 但是它们能更好地达到更高的指令级并行度。

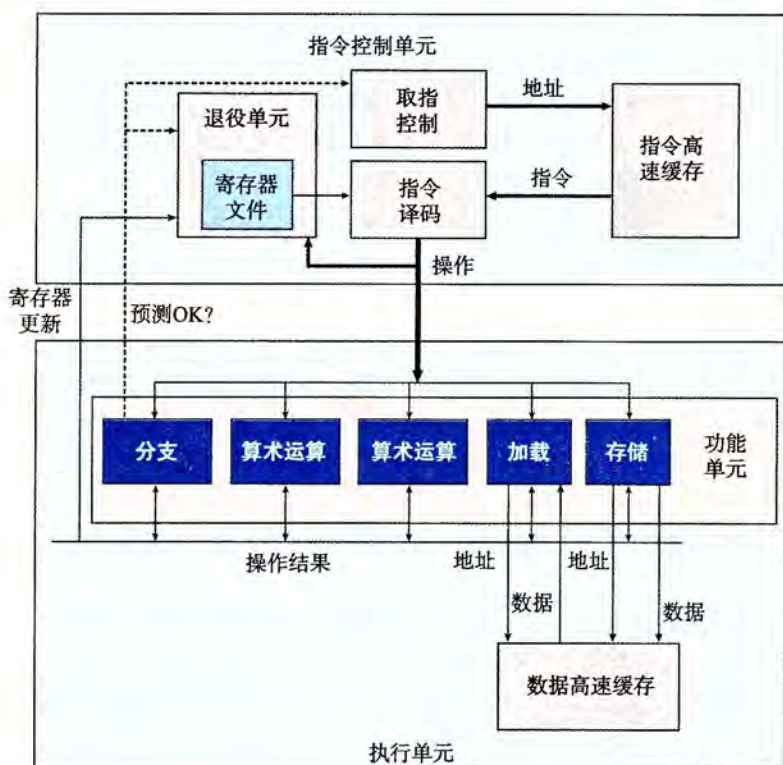


图 5-11 一个乱序处理器的框图。指令控制单元负责从内存中读出指令，并产生一系列基本操作。然后执行单元完成这些操作，以及指出分支预测是否正确

ICU 从指令高速缓存(instruction cache)中读取指令，指令高速缓存是一个特殊的高速存储器，它包含最近访问的指令。通常，ICU 会在当前正在执行的指令很早之前取指，这样它才有足够的时间对指令译码，并把操作发送到 EU。不过，一个问题是当程序遇到分支^①时，程序有两个可能的前进方向。一种可能会选择分支，控制被传递到分支目标。另一种可能是，不选择分支，控制被传递到指令序列的下一条指令。现代处理器采用了一种称为分支预测(branch prediction)的技术，处理器会猜测是否会选择分支，同时还预测分支的目标地址。使用投机执行(speculative execution)的技术，处理器会开始取出位于它预测的分支会跳到的地方的指令，并对指令译码，甚至在它确定分支预测是否正确之前就开始执行这些操作。如果过后确定分支预测错误，会将状态重新设置到分支点的状态，并开始取出和执行另一个方向上的指令。标记为取指控制的块包括分支预测，以完成确定取哪些指令的任务。

指令译码逻辑接收实际的程序指令，并将它们转换成一组基本操作(有时称为微操作)。每个这样的操作都完成某个简单的计算任务，例如两个数相加，从内存中读数据，或是向内存写数据。对于具有复杂指令的机器，比如 x86 处理器，一条指令可以被译码成多个操作。关于指令如何被译码成操作序列的细节，不同的机器都会不同，这个信息可谓是高度机密。幸运的是，不需要知道某台机器实现的底层细节，我们也能优化自己的程序。

① 术语“分支”专指条件转移指令。对处理器来说，其他可能将控制传送到多个目的地址的指令，例如过程返回和间接跳转，带来的也是类似的挑战。

在一个典型的 x86 实现中，一条只对寄存器操作的指令，例如

```
addq %rax,%rdx
```

会被转化成三个操作。另一方面，一条包括一个或者多个内存引用的指令，例如

```
addq %rax,8(%rdx)
```

会产生多个操作，把内存引用和算术运算分开。这条指令会被译码成为三个操作：一个操作从内存中加载一个值到处理器中，一个操作将加载进来的值加上寄存器 `%rax` 中的值，而一个操作将结果存回到内存。这种译码逻辑对指令进行分解，允许任务在一组专门的硬件单元之间进行分割。这些单元可以并行地执行多条指令的不同部分。

EU 接收来自取指单元的操作。通常，每个时钟周期会接收多个操作。这些操作会被分派到一组功能单元中，它们会执行实际的操作。这些功能单元专门用来处理不同类型的操作。

读写内存是由加载和存储单元实现的。加载单元处理从内存读数据到处理器的操作。这个单元有一个加法器来完成地址计算。类似，存储单元处理从处理器写数据到内存的操作。它也有一个加法器来完成地址计算。如图中所示，加载和存储单元通过数据高速缓存 (data cache) 来访问内存。数据高速缓存是一个高速存储器，存放着最近访问的数据值。

使用投机执行技术对操作求值，但是最终结果不会存放在程序寄存器或数据内存中，直到处理器能确定应该实际执行这些指令。分支操作被送到 EU，不是确定分支该往哪里去，而是确定分支预测是否正确。如果预测错误，EU 会丢弃分支点之后计算出来的结果。它还会发信号给分支单元，说预测是错误的，并指出正确的分支目的。在这种情况下，分支单元开始在新的位置取指。如在 3.6.6 节中看到的，这样的预测错误会导致很大的性能开销。在可以取出新指令、译码和发送到执行单元之前，要花费一点时间。

图 5-11 说明不同的功能单元被设计来执行不同的操作。那些标记为执行“算术运算”的单元通常是专门用来执行整数和浮点数操作的不同组合。随着时间的推移，在单个微处理器芯片上能够集成的晶体管数量越来越多，后续的微处理器型号都增加了功能单元的数量以及每个单元能执行的操作组合，还提升了每个单元的性能。由于不同程序间所要求的操作变化很大，因此，算术运算单元被特意设计成能够执行各种不同的操作。比如，有些程序也许会涉及整数操作，而其他则要求许多浮点操作。如果一个功能单元专门执行整数操作，而另一个只能执行浮点操作，那么，这些程序就没有一个能够完全得到多个功能单元带来的好处了。

举个例子，我们的 Intel Core i7 Haswell 参考机有 8 个功能单元，编号为 0~7。下面部分列出了每个单元的功能：

- 0: 整数运算、浮点乘、整数和浮点数除法、分支
- 1: 整数运算、浮点加、整数乘、浮点乘
- 2: 加载、地址计算
- 3: 加载、地址计算
- 4: 存储
- 5: 整数运算
- 6: 整数运算、分支
- 7: 存储、地址计算

在上面的列表中，“整数运算”是指基本的操作，比如加法、位级操作和移位。乘法

和除法需要更多的专用资源。我们看到存储操作要两个功能单元——一个计算存储地址，一个实际保存数据。5.12 节将讨论存储(和加载)操作的机制。

我们可以看出功能单元的这种组合具有同时执行多个同类型操作的潜力。它有 4 个功能单元可以执行整数操作，2 个单元能执行加载操作，2 个单元能执行浮点乘法。稍后我们将看到这些资源对程序获得最大性能所带来的影响。

在 ICU 中，退役单元(retirement unit)记录正在进行的处理，并确保它遵守机器级程序的顺序语义。我们的图中展示了一个寄存器文件，它包含整数、浮点数和最近的 SSE 和 AVX 寄存器，是退役单元的一部分，因为退役单元控制这些寄存器的更新。指令译码时，关于指令的信息被放置在一个先进先出的队列中。这个信息会一直保持在队列中，直到发生以下两个结果中的一个。首先，一旦一条指令的操作完成了，而且所有引起这条指令的分支点也都被确认为预测正确，那么这条指令就可以退役(retired)了，所有对程序寄存器的更新都可以被实际执行了。另一方面，如果引起该指令的某个分支点预测错误，这条指令会被清空(flushed)，丢弃所有计算出来的结果。通过这种方法，预测错误就不会改变程序的状态了。

正如我们已经描述的那样，任何对程序寄存器的更新都只会在指令退役时才会发生，只有在处理器能够确信导致这条指令的所有分支都预测正确了，才会这样做。为了加速一条指令到另一条指令的结果的传送，许多此类信息是在执行单元之间交换的，即图中的“操作结果”。如图中的箭头所示，执行单元可以直接将结果发送给彼此。这是 4.5.5 节中简单处理器设计中采用的数据转发技术的更复杂精细版本。

控制操作数在执行单元间传送的最常见的机制称为寄存器重命名(register renaming)。当一条更新寄存器 r 的指令译码时，产生标记 t ，得到一个指向该操作结果的唯一的标识符。条目 (r, t) 被加入到一张表中，该表维护着每个程序寄存器 r 与会更新该寄存器的操作的标记 t 之间的关联。当随后以寄存器 r 作为操作数的指令译码时，发送到执行单元的操作会包含 t 作为操作数源的值。当某个执行单元完成第一个操作时，会生成一个结果 (v, t) ，指明标记为 t 的操作产生值 v 。所有等待 t 作为源的操作都能使用 v 作为源值，这就是一种形式的数据转发。通过这种机制，值可以从一个操作直接转发到另一个操作，而不是写到寄存器文件再读出来，使得第二个操作能够在第一个操作完成后尽快开始。重命名表只包含关于有未进行写操作的寄存器条目。当一条被译码的指令需要寄存器 r ，而又没有标记与这个寄存器相关联，那么可以直接从寄存器文件中获取这个操作数。有了寄存器重命名，即使只有在处理器确定了分支结果之后才能更新寄存器，也可以预测着执行操作的整个序列。

旁注 乱序处理的历史

乱序处理最早是在 1964 年 Control Data Corporation 的 6600 处理器中实现的。指令由十个不同的功能单元处理，每个单元都能独立地运行。在那个时候，这种时钟频率为 10Mhz 的机器被认为是科学计算最好的机器。

在 1966 年，IBM 首先是在 IBM 360/91 上实现了乱序处理，但只是用来执行浮点指令。在大约 25 年的时间里，乱序处理都被认为是一项异乎寻常的技术，只在追求尽可能高性能的机器中使用，直到 1990 年 IBM 在 RS/6000 系列工作站中重新引入了这项技术。这种设计成为了 IBM/Motorola PowerPC 系列的基础，1993 年引入的型号 601，它成为第一个使用乱序处理的单芯片微处理器。Intel 在 1995 年的 PentiumPro 型号中引入了乱序处理，PentiumPro 的底层微体系结构类似于我们的参考机。

5.7.2 功能单元的性能

图 5-12 提供了 Intel Core i7 Haswell 参考机的一些算术运算的性能, 有的是测量出来的, 有的是引用 Intel 的文献[49]。这些时间对于其他处理器来说也是具有代表性的。每个运算都是由以下这些数值来刻画的: 一个是延迟(latency), 它表示完成运算所需要的总时间; 另一个是发射时间(issue time), 它表示两个连续的同类型的运算之间需要的最小时钟周期数; 还有一个是容量(capacity), 它表示能够执行该运算的功能单元的数量。

运算	整数			浮点数		
	延迟	发射	容量	延迟	发射	容量
加法	1	1	4	3	1	1
乘法	3	1	1	5	1	2
除法	3 ~ 30	3 ~ 30	1	3 ~ 15	3 ~ 15	1

图 5-12 参考机的操作的延迟、发射时间和容量特性。延迟表明执行实际运算所需要的时钟周期总数, 而发射时间表明两次运算之间间隔的最小周期数。容量表明同时能发射多少个这样的操作。除法需要的时间依赖于数据值

我们看到, 从整数运算到浮点运算, 延迟是增加的。还可以看到加法和乘法运算的发射时间都为 1, 意思是说在每个时钟周期, 处理器都可以开始一条新的这样的运算。这种很短的发射时间是通过使用流水线实现的。流水线化的功能单元实现为一系列的阶段(stage), 每个阶段完成一部分的运算。例如, 一个典型的浮点加法器包含三个阶段(所以有三个周期的延迟): 一个阶段处理指数值, 一个阶段将小数相加, 而另一个阶段对结果进行舍入。算术运算可以连续地通过各个阶段, 而不用等待一个操作完成后再开始下一个。只有当要执行的运算是连续的、逻辑上独立的时候, 才能利用这种功能。发射时间为 1 的功能单元被称为完全流水线化的(fully pipelined): 每个时钟周期可以开始一个新的运算。出现容量大于 1 的运算是由于有多个功能单元, 就如前面所述的参考机一样。

我们还看到, 除法器(用于整数和浮点除法, 还用来计算浮点平方根)不是完全流水线化的——它的发射时间等于它的延迟。这就意味着在开始一条新运算之前, 除法器必须完成整个除法。我们还看到, 对于除法的延迟和发射时间是以范围的形式给出的, 因为某些被除数和除数的组合比其他的组合需要更多的步骤。除法的长延迟和长发射时间使之成为了一个相对开销很大的运算。

表达发射时间的一种更常见的方法是指明这个功能单元的最大吞吐量, 定义为发射时间的倒数。一个完全流水线化的功能单元有最大的吞吐量, 每个时钟周期一个运算, 而发射时间较大的功能单元的最大吞吐量比较小。具有多个功能单元可以进一步提高吞吐量。对一个容量为 C , 发射时间为 I 的操作来说, 处理器可能获得的吞吐量为每时钟周期 C/I 个操作。比如, 我们的参考机可以每个时钟周期执行两个浮点乘法运算。我们将看到如何利用这种能力来提高程序的性能。

电路设计者可以创建具有各种性能特性的功能单元。创建一个延迟短或使用流水线的单元需要较多的硬件, 特别是对于像乘法和浮点操作这样比较复杂的功能。因为微处理器芯片上, 对于这些单元, 只有有限的空间, 所以 CPU 设计者必须小心地平衡功能单元的数量和它们各自的性能, 以获得最优的整体性能。设计者们评估许多不同的基准程序, 将大多数资源用于最关键的操作。如图 5-12 表明的那样, 在 Core i7 Haswell 处理器的设计中, 整数乘法、浮点乘法和加法被认为是重要的操作, 即使为了获得低延迟和较高的流水

线化程度需要大量的硬件。另一方面，除法相对不太常用，而且要想实现低延迟或完全流水线化是很困难的。

这些算术运算的延迟、发射时间和容量会影响合并函数的性能。我们用 CPE 值的两个基本界限来描述这种影响：

界限	整数		浮点数	
	+	*	+	*
延迟	1.00	3.00	3.00	5.00
吞吐量	0.50	1.00	1.00	0.50

延迟界限给出了任何必须按照严格顺序完成合并运算的函数所需要的最小 CPE 值。根据功能单元产生结果的最大速率，吞吐量界限给出了 CPE 的最小界限。例如，因为只有一个整数乘法器，它的发射时间为 1 个时钟周期，处理器不可能支持每个时钟周期大于 1 条乘法的速度。另一方面，四个功能单元都可以执行整数加法，处理器就有可能持续每个周期执行 4 个操作的速率。不幸的是，因为需要从内存读数据，这造成了另一个吞吐量界限。两个加载单元限制了处理器每个时钟周期最多只能读取两个数据值，从而使得吞吐量界限为 0.50。我们会展示延迟界限和吞吐量界限对合并函数不同版本的影响。

5.7.3 处理器操作的抽象模型

作为分析在现代处理器上执行的机器级程序性能的一个工具，我们会使用程序的数据流(data-flow)表示，这是一种图形化的表示方法，展现了不同操作之间的数据相关是如何限制它们的执行顺序的。这些限制形成了图中的关键路径(critical path)，这是执行一组机器指令所需时钟周期数的一个下界。

在继续技术细节之前，检查一下函数 `combine4` 的 CPE 测量值是很有帮助的，到目前为止 `combine4` 是最快的代码：

函数	方法	整数		浮点数	
		+	*	+	*
<code>combine4</code>	累积在临时变量中	1.27	3.01	3.01	5.01
延迟界限		1.00	3.00	3.00	5.00
吞吐量界限		0.50	1.00	1.00	0.50

我们可以看到，除了整数加法的情况，这些测量值与处理器的延迟界限是一样的。这不是巧合——它表明这些函数的性能是由所执行的求和或者乘积计算主宰的。计算 n 个元素的乘积或者和需要大约 $L \cdot n + K$ 个时钟周期，这里 L 是合并运算的延迟，而 K 表示调用函数和初始化以及终止循环的开销。因此，CPE 就等于延迟界限 L 。

1. 从机器级代码到数据流图

程序的数据流表示是非正式的。我们只是想用它来形象地描述程序中的数据相关是如何主宰程序的性能的。以 `combine4`(图 5-10)为例来描述数据流表示法。我们将注意力集中在循环执行的计算上，因为对于大向量来说，这是决定性能的主要因素。我们考虑类型为 `double` 的数据、以乘法作为合并运算的情况，不过其他数据类型和运算的组合也有几乎一样的结构。这个循环编译出的代码由 4 条指令组成，寄存器 `%rdx` 存放指向数组 `data` 中第 i 个元素的指针，`%rax` 存放指向数组末尾的指针，而 `%xmm0` 存放累积值 `acc`。


```

Inner loop of combine4. data_t = double, OP = *
acc in %xmm0, data+i in %rdx, data+length in %rax
1  .L25:                                loop:
2  vmulsd (%rdx), %xmm0, %xmm0      Multiply acc by data[i]
3  addq    $8, %rdx                Increment data+i
4  cmpq    %rax, %rdx              Compare to data+length
5  jne     .L25                    If !=, goto loop

```

如图 5-13 所示，在我们假想的处理器设计中，指令译码器会把这 4 条指令扩展成一系列的三步操作，最开始的乘法指令被扩展成一个 load 操作，从内存读出源操作数，和一个 mul 操作，执行乘法。

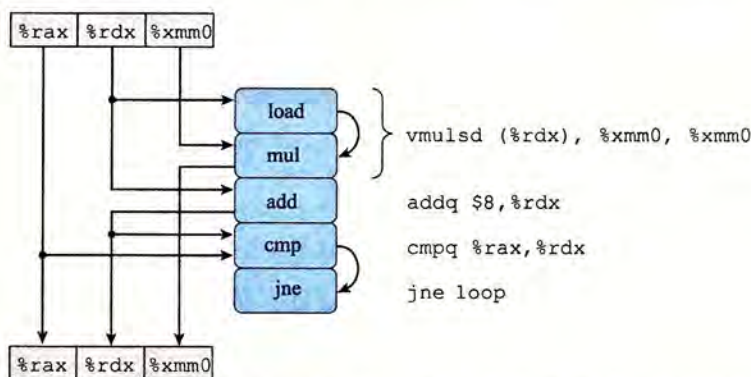


图 5-13 combine4 的内循环代码的图形化表示。指令动态地被翻译成一个或两个操作，每个操作从其他操作或寄存器接收值，并且为其他操作和寄存器产生值。我们给出最后一条指令的目标为标号 loop。它跳转到给出的第一条指令

作为生成程序数据流图表示的一步，图 5-13 左手边的方框和线给出了各个指令是如何使用和更新寄存器的，顶部的方框表示循环开始时寄存器的值，而底部的方框表示最后寄存器的值。例如，寄存器 `%rax` 只被 `cmp` 操作作为源值，因此这个寄存器在循环结束时有着同循环开始时一样的值。另一方面，在循环中，寄存器 `%rdx` 既被使用也被修改。它的初始值被 `load` 和 `add` 操作使用；它的新值由 `add` 操作产生，然后被 `cmp` 操作使用。在循环中，`mul` 操作首先使用寄存器 `%xmm0` 的初始值作为源值，然后会修改它的值。

图 5-13 中的某些操作产生的值不对应于任何寄存器。在右边，用操作间的弧线来表示。`load` 操作从内存读出一个值，然后把它直接传递到 `mul` 操作。由于这两个操作是通过一条 `vmulsd` 指令译码产生的，所以这个在两个操作之间传递的中间值没有与之相关联的寄存器。`cmp` 操作更新条件码，然后 `jne` 操作会测试这些条件码。

对于形成循环的代码片段，我们可以将访问到的寄存器分为四类：

只读：这些寄存器只用作源值，可以作为数据，也可以用来计算内存地址，但是在循环中它们是不会被修改的。循环 combine4 的只读寄存器是 `%rax`。

只写：这些寄存器作为数据传送操作的目的。在本循环中没有这样的寄存器。

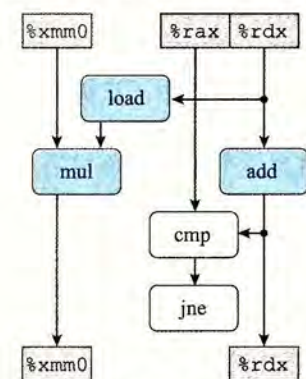
局部：这些寄存器在循环内部被修改和使用，迭代与迭代之间不相关。在这个循环中，条件码寄存器就是例子：`cmp` 操作会修改它们，然后 `jne` 操作会使用它们，不过这种相关是在单次迭代之内的。

循环：对于循环来说，这些寄存器既作为源值，又作为目的，一次迭代中产生的值会在另一次迭代中用到。可以看到，`%rdx` 和 `%xmm0` 是 combine4 的循环寄存器，对应于程序

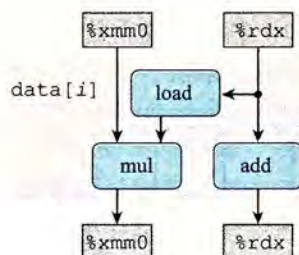
值 $\text{data}+i$ 和 acc 。

正如我们会看到的，循环寄存器之间的操作链决定了限制性能的数据相关。

图 5-14 是对图 5-13 的图形化表示的进一步改进，目标是只给出影响程序执行时间的操作和数据相关。在图 5-14a 中看到，我们重新排列了操作符，更清晰地表明了从顶部源寄存器（只读寄存器和循环寄存器）到底部目的寄存器（只写寄存器和循环寄存器）的数据流。



a) 重新排列了图 5-13 的操作符，更清晰地表明了数据相关



b) 操作在一次迭代中使用某些值，产生出在下一迭代中需要的新值

图 5-14 将 combine4 的操作抽象成数据流图

在图 5-14a 中，如果操作符不属于某个循环寄存器之间的相关链，那么就把它标识成白色。例如，比较 (cmp) 和分支 (jne) 操作不直接影响程序中的数据流。假设指令控制单元预测会选择分支，因此程序会继续循环。比较和分支操作的目的是测试分支条件，如果不选择分支的话，就通知 ICU。我们假设这个检查能够完成得足够快，不会减慢处理器的执行。

在图 5-14b 中，消除了左边标识为白色的操作符，而且只保留了循环寄存器。剩下的是一个抽象的模板，表明的是由于循环的一次迭代在循环寄存器中形成的数据相关。在这个图中可以看到，从一次迭代到下一次迭代有两个数据相关。在一边，我们看到存储在寄存器 $\text{\%xmm0}$ 中的程序值 acc 的连续的值之间有相关。通过将 acc 的旧值乘以一个数据元素，循环计算出 acc 的新值，这个数据元素是由 load 操作产生的。在另一边，我们看到循环索引 i 的连续的值之间有相关。每次迭代中， i 的旧值用来计算 load 操作的地址，然后 add 操作也会增加它的值，计算出新值。

图 5-15 给出了函数 combine4 内循环的 n 次迭代的数据流表示。可以看出，简单地重

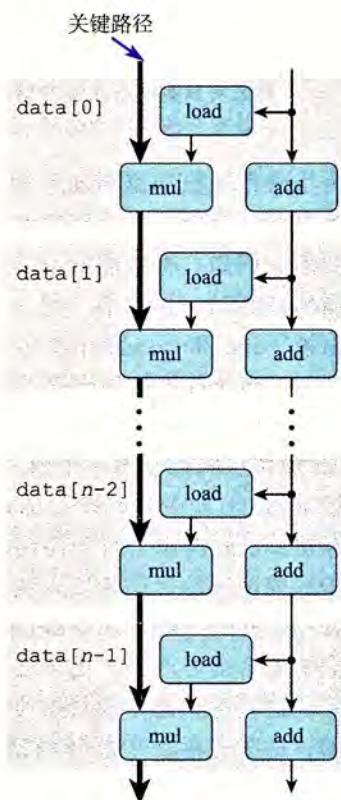


图 5-15 combine4 的内循环的 n 次迭代计算的数据流表示。乘法操作的序列形成了限制程序性能的关键路径

复图 5-14 右边的模板 n 次, 就能得到这张图。我们可以看到, 程序有两条数据相关链, 分别对应于操作 mul 和 add 对程序值 acc 和 data+i 的修改。假设浮点乘法延迟为 5 个周期, 而整数加法延迟为 1 个周期, 可以看到左边的链会成为关键路径, 需要 $5n$ 个周期执行。右边的链只需要 n 个周期执行, 因此, 它不会制约程序的性能。

图 5-15 说明在执行单精度浮点乘法时, 对于 combine4, 为什么我们获得了等于 5 个周期延迟界限的 CPE。当执行这个函数时, 浮点乘法器成为了制约资源。循环中需要的其他操作——控制和测试指针值 data+i, 以及从内存中读数据——与乘法器并行地进行。每次后继的 acc 的值被计算出来, 它就反馈回来计算下一个值, 不过只有等到 5 个周期后才能完成。

其他数据类型和运算组合的数据流与图 5-15 所示的内容一样, 只是在左边的形成数据相关链的数据操作不同。对于所有情况, 如果运算的延迟, L 大于 1, 那么可以看到测量出来的 CPE 就是 L , 表明这个链是制约性能的关键路径。

2. 其他性能因素

另一方面, 对于整数加法的情况, 我们对 combine4 的测试表明 CPE 为 1.27, 而根据沿着图 5-15 中左边和右边形成的相关链预测的 CPE 为 1.00, 测试值比预测值要慢。这说明了一个原则, 那就是数据流表示中的关键路径提供的只是程序需要周期数的下界。还有其他一些因素会限制性能, 包括可用的功能单元的数量和任何一步中功能单元之间能够传递数据值的数量。对于合并运算为整数加法的情况, 数据操作足够快, 使得其他操作供应数据的速度不够快。要准确地确定为什么程序中每个元素需要 1.27 个周期, 需要比公开可以获得的更详细的硬件设计知识。

总结一下 combine4 的性能分析: 我们对程序操作的抽象数据流表示说明, combine4 的关键路径长 $L \cdot n$ 是由对程序值 acc 的连续更新造成的, 这条路径将 CPE 限制为最多 L 。除了整数加法之外, 对于所有的其他情况, 测量出的 CPE 确实等于 L , 对于整数加法, 测量出的 CPE 为 1.27 而不是根据关键路径的长度所期望的 1.00。

看上去, 延迟界限是基本的限制, 决定了我们的合并运算能执行多快。接下来的任务是重新调整操作的结构, 增强指令级并行性。我们想对程序做变换, 使得唯一的限制变成吞吐量界限, 得到接近于 1.00 的 CPE。

 **练习题 5.5** 假设写一个对多项式求值的函数, 这里, 多项式的次数为 n , 系数为 $a_0, a_1, \dots, a_n$ 。对于值 x , 我们对多项式求值, 计算

$$a_0 + a_1x + a_2x^2 + \dots + a_nx^n \quad (5.2)$$

这个求值可以用下面的函数来实现, 参数包括一个系数数组 a、值 x 和多项式的次数 degree(等式(5.2)中的值 n)。在这个函数的一个循环中, 我们计算连续的等式的项, 以及连续的 x 的幂:

```
1 double poly(double a[], double x, long degree)
2 {
3     long i;
4     double result = a[0];
5     double xpwr = x; /* Equals x~i at start of loop */
6     for (i = 1; i <= degree; i++) {
7         result += a[i] * xpwr;
8         xpwr = x * xpwr;
9     }
10    return result;
11 }
```


- A. 对于次数 n ，这段代码执行多少次加法和多少次乘法运算？
- B. 在我们的参考机上，算术运算的延迟如图 5-12 所示，我们测量了这个函数的 CPE 等于 5.00。根据由于实现函数第 7~8 行的操作迭代之间形成的数据相关，解释为什么会得到这样的 CPE。

 **练习题 5.6** 我们继续探索练习题 5.5 中描述的多项式求值的方法。通过采用 Horner 法(以英国数学家 William G. Horner(1786—1837)命名)对多项式求值，我们可以减少乘法的数量。其思想是反复提出 x 的幂，得到下面的求值：

$$a_0 + x(a_1 + x(a_2 + \cdots + x(a_{n-1} + xa_n) \cdots)) \quad (5.3)$$

使用 Horner 法，我们可以用下面的代码实现多项式求值：

```
1  /* Apply Horner's method */
2  double polyh(double a[], double x, long degree)
3  {
4      long i;
5      double result = a[degree];
6      for (i = degree-1; i >= 0; i--)
7          result = a[i] + x*result;
8      return result;
9  }
```

- A. 对于次数 n ，这段代码执行多少次加法和多少次乘法运算？
- B. 在我们的参考机上，算术运算的延迟如图 5-12 所示，测量这个函数的 CPE 等于 8.00。根据由于实现函数第 7 行的操作迭代之间形成的数据相关，解释为什么会得到这样的 CPE。
- C. 请解释虽然练习题 5.5 中所示的函数需要更多的操作，但是它是如何运行得更快的。

5.8 循环展开

循环展开是一种程序变换，通过增加每次迭代计算的元素的数量，减少循环的迭代次数。psum2 函数(见图 5-1)就是这样一个例子，其中每次迭代计算前置和的两个元素，因而将需要的迭代次数减半。循环展开能够从两个方面改进程序的性能。首先，它减少了不直接有助于程序结果的操作的数量，例如循环索引计算和条件分支。第二，它提供了一些方法，可以进一步变化代码，减少整个计算中关键路径上的操作数量。在本节中，我们会看一些简单的循环展开，不做任何进一步的变化。

图 5-16 是合并代码的使用“ 2×1 循环展开”的版本。第一个循环每次处理数组的两个元素。也就是每次迭代，循环索引 i 加 2，在一次迭代中，对数组元素 i 和 $i+1$ 使用合并运算。

一般来说，向量长度不一定是 2 的倍数。想要使我们的代码对任意向量长度都能正确工作，可以从两个方面来解释这个需求。首先，要确保第一次循环不会超出数组的界限。对于长度为 n 的向量，我们将循环界限设为 $n-1$ 。然后，保证只有当循环索引 i 满足 $i < n-1$ 时才会执行这个循环，因此最大数组索引 $i+1$ 满足 $i+1 < (n-1)+1 = n$ 。

把这个思想归纳为对一个循环按任意因子 k 进行展开，由此产生 $k \times 1$ 循环展开。为此，上限设为 $n-k+1$ ，在循环内对元素 i 到 $i+k-1$ 应用合并运算。每次迭代，循环索引 i 加 k 。那么最大循环索引 $i+k-1$ 会小于 n 。要使用第二个循环，以每次处理一个元素的方式处理向量的最后几个元素。这个循环体将会执行 $0 \sim k-1$ 次。对于 $k=2$ ，我们能用一个简单的条件语句，可选地增加最后一次迭代，如函数 psum2(图 5-1)所示。对于 $k>2$ ，最后的这些情